

Auditoria Técnica e Funcional — ERPOS V2

Escopo: PDV Caixa, Autoatendimento, Mesa QR Code, Gestão de Pedidos, Relatórios, Estoque, Configurações relacionadas, Diagnóstico/QA. **Modo:** somente leitura — nenhum arquivo ou dado foi alterado. Inclui verificação direta das funções SQL (RPCs) no projeto Supabase "ERP OS" (mdghhj emz dmeu qpz uyz x).

1. Resumo Executivo

O sistema **NÃO está pronto para produção** nos módulos analisados. A camada de frontend é robusta e bem estruturada (proteção de duplo clique em `useOrderSubmit`, fila de impressão com `retry`, cálculo de SLA, etc.), mas existem **gaps críticos na camada de cancelamento/estorno/restock**, uma **funcionalidade de estorno totalmente falsa no PDV Caixa**, **risco real de duplicidade de numeração de pedidos entre sessões simultâneas**, e **dois fluxos paralelos e divergentes de pedido de mesa** (Mesa QR vs Mesa "garçom do cliente").

Resumo por módulo:

Módulo	Situação	Bloqueadores críticos
PDV Caixa	Funcional, mas com estorno fake e múltiplos caminhos paralelos de cancelamento	BUG-001, BUG-002, BUG-005
Autoatendimento	Funcional, mas modo offline não persiste pedido	BUG-025, BUG-026
Mesa QR / Mesa Cliente	Dois sistemas paralelos divergentes para pedido de mesa	BUG-018, BUG-021
Gestão de Pedidos / KDS	Status "Em Rota" não persiste; cancelamento sem estorno/restock/auditoria	BUG-005, BUG-010, BUG-012
Relatórios	CMV duplicado/divergente; fallback de caixa zera valores	BUG-037, BUG-039/040
Estoque	Estoque não é devolvido em cancelamento; lotes não são baixados nas vendas	BUG-005, BUG-008
Numeração de pedidos	Risco de duplicidade entre sessões simultâneas no mesmo dia	BUG-016
Diagnóstico/QA	Simulações cobrem "caminho feliz", mas não cobrem nenhum dos bugs acima	BUG-043 a BUG-052

Veredito: o "núcleo" financeiro (criar pedido, pagar, fechar caixa, baixar estoque na venda) está sólido e testado. O **elo fraco é tudo que envolve cancelamento, estorno, reembolso e restock** — múltiplas implementações divergentes, nenhuma delas completa. Isso é o que precisa ser resolvido antes de operar com cliente real, pois envolve dinheiro e estoque diretamente.

2. Mapa da Arquitetura Analisada

```
PDV Caixa (src/pages/pdv/caixa) ■■■ PDVContext / SessaoContext ■■■ order-write
(create/finalize/advance/cancel/refund/payments) ■ ■■ fn_create_order_bypass /
fn_create_order_items_bypass ■ ■■ fn_record_payment_bypass ■ ■■ fn_next_order_number(session_id[,
tenant_id]) → sessions.last_order_number ■ ■■ fn_advance_order_status_safe(order_id, status) →
orders/order_items ■ ■■ fn_cancel_order_bypass / fn_cancel_and_refund_order / fn_restock_order* ■
■■■ deductStockForOrderItem() → fn_add_stock_movement / fn_update_ingredient_stock ■■
fn_open_cash_register / fn_close_cash_register_v2 / add_cash_movement (RPCs via SessaoContext) ■■
Impressão: sendToPrinter() (direto, sem fila) | print_queue + agente-local (com fila/retry)
Autoatendimento (src/pages/autoatendimento) ■■ KioskAuthContext (kiosk-auth) + useOrderSubmit →
```

order-write (mesmo backend do caixa) Mesa QR (src/pages/mesa-qr) ■■ useMesaQRData → mesa-write (create_mesa_order, lookup_mesa, get_meus_pedidos...) → orders (origin_type='table', destination_type='password', participant_id/access_token) Mesa "Cliente" / Garçon do cliente (src/pages/mesa) ■■ MesasContext / MesaEdicaoContext → order-write (create_order direto) + table-write (status/fechamento) → orders (origin_type='table', destination_name=nome livre) – MODELO DIFERENTE do Mesa QR acima Gestão de Pedidos (src/pages/gestor-pedidos, src/pages/pedidos) ■■ KDSContext / ProducaoContext → leitura orders/order_items + order-write (cancel/advance) Relatórios (src/pages/relatorios) ■■ useSalesReport / useOrigemReport / useCaixaReport / useCmvRelatorio / useDeliveryReport / useCancelamentosReport → fn_get_sales_report, fn_get_cash_sessions_v2, fn_get_cmv_report, fn_get_cancelamentos_report (RPCs) Estoque (src/pages/estoque) ■■ EstoqueContext → stock-write (add_stock_movement, create_batch, confirm_inventory, cmv) → fn_add_stock_movement / fn_update_ingredient_stock / ingredient_batches ■■ Produção → production-write → fn_register_production_and_stock_v2 Diagnóstico/QA (src/pages/diagnostico) ■■ qa-full-simulation (tenant QA isolado) | simulate-orders / simulate-pdv-orders (tenant de PRODUÇÃO real)

3. Matriz de Simulações (consolidada)

Legenda de Status: ■ Aprovado · ■ Reprovado · ■■ Precisa validar manualmente

PDV Caixa / Pagamentos / Caixa

#	Cenário	Resultado encontrado	Status
1	Abertura de caixa	fn_open_cash_register cria registro corretamente, auto-cria usuário se necessário	■
2	Venda simples	fn_create_order_bypass + fn_create_order_items_bypass + fn_record_payment_bypass consistentes	■
3	Venda com adicional (opção com ingredient_id/production_recipe_id)	Dedução de estoque resolve recompensas	■
4	Venda com observação	OK, snapshot salvo em order_item_observacoes/campo livre	■
5	Venda com combo	buildDeductions cobre combo_ingredients e fallback combo_items→item_ingredients	■
6	Venda com desconto	Persistido em order_discounts/discount_amount; mas ver BUG-039/040 no relatório	■■
7	Dinheiro com troco	payments.change_amount calculado e somado	■
8	PIX	Fluxo de criação de pagamento OK; integração Stone real não implementada (fora do escopo desta rodada)	■■
9	Cartão	OK	■
10	Múltiplas formas de pagamento	payment_group_id agrupa pagamentos de pedidos vinculados	■

11	Sangria	cash_movements type='out' → fn_close_cash_register_v2 soma corretamente	■
12	Suprimento	cash_movements type='in' → idem	■
13	Cancelamento ANTES do pagamento	fn_cancel_order_bypass/order-write.cancel_order cancela order+items; sem payments a reverter — OK	■
14	Cancelamento DEPOIS do pagamento	Reembolso só ocorre se reason_type+refund_amount+refund_method forem enviados; restock NUNCA ocorre mesmo quando restock_items=true	■ BUG-002/005
15	Estorno (tela "Estornar" do PDV)	EstornoModal não chama nenhuma RPC — é 100% simulado no frontend, com senha hardcoded 1234	■ BUG-001
16	Fechamento de caixa	fn_close_cash_register_v2: expected = opening + cash_payments - withdrawals + deposits; fórmula correta e verificada no banco	■
17	Pedido pendente no fechamento	Não há bloqueio/aviso explícito de pedidos new/preparing abertos ao fechar sessão	■■ BUG-017
18	Impressora offline	Caminho direto (sendToPrinter, usado pela comanda de cozinha) não tem fila/retry; só a fila central (print_queue) tem retry, mas não é usada pela comanda principal	■ BUG-028/029
19	Duplo clique em finalizar pedido	useOrderSubmit usa submittingRef — protegido	■
20	Produto sem estoque	Apenas alerta visual (fn_check_stock_alert_for_items), não bloqueia. Estoque clamped em 0 (não fica negativo) pelas RPCs fn_update_ingredient_stock/fn_add_stock_movement	■■ BUG-006
21	Produto inativo	Item inativo não aparece no grid (frontend filtra) — não testado server-side se order-write rejeita item inativo enviado manualmente	■■

Autoatendimento

#	Cenário	Resultado encontrado	Status
1	Abrir cardápio / adicionar / remover / alterar quantidade	OK	■
2	Finalizar pedido	OK via useOrderSubmit → order-write	■

3	Carrinho vazio	Bloqueado no frontend antes de chamar <code>order-write</code>	■
4	Perda de conexão no pagamento	<code>useOrderSubmit</code> faz retry com backoff e fallback IndexedDB (offline queue)	■
5	Duplo clique em finalizar	Protegido por <code>submittingRef</code>	■
6	Modo "sem sessão" (! <code>sessionId</code>)	Pedido só é salvo localmente (<code>addPedido/KDS</code> local), nunca persiste em <code>orders/order_items</code>. Some no refresh	■ BUG-025
7	Fallback de formas de pagamento (DB indisponível)	IDs ' <code>pix</code> '/' <code>credito</code> '/... não são UUID → falha em <code>record_payment</code> (FK)	■ BUG-026
8	Sessão de caixa fecha durante o pedido	Sem tratamento específico do erro 403 <code>session_closed</code>	■■ BUG-027
9	Produto fica inativo durante o pedido / sem estoque	Mesmo comportamento do PDV (alerta, não bloqueia)	■■
10	Pedido chega na gestão/KDS	OK, mesmo <code>order-write</code> do PDV	■

Mesa QR Code / Mesa Cliente

#	Cenário	Resultado encontrado	Status
1	Acesso via QR Code	<code>mesa-write.lookup_mesa</code> por <code>qr_token</code>	■
2	Vínculo pedido↔mesa	Via <code>participant_id/table_session_id</code> (Mesa QR) — mas a tela /<code>mesa/:mesaId</code> (Mesa Cliente) cria pedidos via <code>order-write</code> direto, sem esses campos	■ BUG-018
3	Dois clientes na mesma mesa	Cada um com <code>participant_id/access_token</code> próprio (Mesa QR) — OK nesse fluxo	■
4	Cliente de uma mesa acessa outra	Token <code>access_token</code> (PIN sequencial) gerado por mesa/sessão — isolamento depende de RLS, não auditado nesta rodada	■■
5	Mesa encerrada com pedido pendente	Dois algoritmos diferentes de "pode fechar": <code>MesasContext.fecharMesa</code> (client-side, <code>payments.is_refunded=false</code>) vs <code>table-write.close_table_by_customer</code> (<code>totalPago >= totalConsumo*0.99</code>)	■ BUG-022
6	Pedido da mesa aparece no gestor	Sim, mas com <code>destination_name</code> em formatos diferentes conforme o fluxo (PIN vs nome livre)	■ BUG-018

7	Status da mesa muda corretamente	Pelo menos 3 vocabulários de status (available/occupied, livre/ocupada/reservada/bloqueada, blocked) coexistem	■ BUG-019
8	Sessão de mesa não fica "zumbi"	Trigger de auto-fechamento existe (mencionado no project_plan), mas o fail-open do fetch de status (BUG-024) pode mascarar sessões já fechadas	■■
9	order-edit-lock / força desbloqueio	force_unlock sem checagem de papel no servidor (qualquer usuário do tenant pode liberar edição de outro)	■ BUG-021

Gestão de Pedidos / KDS

#	Cenário	Resultado encontrado	Status
1	Pedido aparece após criação	OK	■
2	Mudança de status (preparing/ready/delivered)	fn_advance_order_status_saf atualiza orders.status e todos os order_items não cancelados para o mesmo status — não diferencia status por item/estação	■ BUG-011
3	Status "Em Rota" (delivery)	Enum order_status no banco NÃO possui em_rota — handleEmRota no frontend só faz setState local, nunca persiste (provavelmente por isso não chama a RPC, que erraria)	■ BUG-010
4	Cancelamento	fn_cancel_order_bypass (sem refund/restock/audit) e fn_cancel_and_refund_order (refund OK, restock nunca implementado apesar do parâmetro p_restock) — dois caminhos, nenhum completo	■ BUG-005/012
5	Histórico de edição (auditoria)	audit_log só recebe order_refunded quando o bloco condicional de refund é executado; cancelamento simples não gera entrada de auditoria	■ BUG-012
6	Filtros/busca/SLA	3 definições de threshold de SLA diferentes entre utils.ts/SlaCell (Cozinha vs Espera)	■■ BUG-014
7	kdsParaRecente	Zera total/preco dos itens (hardcoded 0)	■ BUG-013
8	Numeração de pedidos	Sequência por sessão (sessions.last_order_number), não por tenant/dia — duas sessões abertas simultaneamente no mesmo dia geram P[ddmmaa]0001 duplicado	■ BUG-016

9	Pedidos de origens diferentes no gestor	OK estruturalmente, mas <code>destination_name</code> tem formatos divergentes (BUG-018)	■■
---	---	--	----

Relatórios

#	Cenário	Resultado encontrado	Status
1	Pedido vendido aparece em vendas	<code>fn_get_sales_report</code> (RPC remota, não auditável via repo, mas é fonte única)	■■
2	Pedido cancelado não soma	<code>fn_get_cash_sessions_v2</code> filtra <code>status NOT IN ('cancelled', 'draft')</code> consistentemente — confirmado no código da RPC	■
3	Desconto reduz total	<code>order_discounts</code> somado corretamente na RPC <code>fn_get_cash_sessions_v2.total_descontos</code>	■ (na RPC)
4	Troco	<code>payments.change_amount</code> somado corretamente na RPC, com <code>is_refunded=false</code>	■ (na RPC)
5	Fallback de <code>useCaixaReport</code> (quando RPC falha)	<code>total_descontos</code> , <code>total_troco</code> , <code>cash_transactions</code> , <code>movimentos.lista</code> ficam zerados/vazios sem aviso ao usuário	■ BUG-039/040
6	CMV (Relatórios vs Estoque)	Duas implementações com fontes de receita diferentes (<code>total_amount</code> vs soma de <code>item_price*qty</code>)	■ BUG-037
7	Relatório por origem (<code>porOrigem</code> vs <code>porHora</code>)	Uma vem de RPC, outra de query direta com filtro próprio — podem divergir	■ BUG-041
8	Timezone	Fixo em <code>America/Sao_Paulo/-03:00</code> , duplicado em <code>useDeliveryReport</code>	■■ BUG-038/042

Estoque

#	Cenário	Resultado encontrado	Status
1	Venda baixa estoque	<code>deductStockForOrderItem</code> → <code>fn_add_stock_movement/fn_update_ingredient_stock</code> , com proteção de duplicidade (<code>ilike reason '%:orderId'</code>)	■
2	Adicional baixa estoque	Resolve <code>ingredient_id</code> direto e via <code>production_recipe_id</code> → <code>output_ingredient_id</code>	■
3	Combo baixa estoque	<code>combo_ingredients</code> + fallback <code>combo_items</code>	■

4	Cancelamento devolve estoque	Nunca, em nenhum dos caminhos (<code>fn_cancel_order_bypass</code> , <code>fn_cancel_and_refund_order</code> , <code>order-write.cancel_order</code>)	■ BUG-005
5	Produto sem estoque bloqueia venda	Não — apenas alerta (fail-safe documentado no próprio código)	■■ BUG-006
6	Estoque negativo	Impossível — <code>GREATEST(0, ...)</code> em <code>fn_update_ingredient_stock</code> e <code>fn_add_stock_movement</code> . Mas isso mascara o tamanho real do déficit (<code>is_depleted=true</code> sem registrar quanto faltou)	■■ BUG-006
7	Inventário/ajuste manual	<code>confirm_inventory</code> chama <code>fn_add_stock_movement</code> (que já atualiza <code>current_stock</code> via <code>GREATEST(0, ...)</code>) e DEPOIS sobrescreve <code>current_stock=qtydContada</code> diretamente — resultado final correto, mas movimento registrado pode não bater com o delta real se <code>fn_add_stock_movement</code> falhar no meio do loop	■ BUG-007
8	Lotes (<code>ingredient_batches.quantity</code>)	<code>fn_add_stock_movement</code> SUPORTA decrementar lote se <code>batch_id</code> for passado — mas nem <code>deductStockForOrderItem</code> (vendas) nem <code>fn_register_production_and_stock_v2</code> (produção) passam <code>batch_id</code> → lotes nunca decrementam na prática	■ BUG-008
9	Produção: custo/rendimento	<code>fn_register_production_and_stock_v2</code> aceita <code>p_total_cost/p_unit_cost</code> calculados no frontend sem revalidação server-side	■■ BUG-009

4. Lista de Bugs

CRÍTICOS

BUG-001 — "Estornar Pedido" no PDV Caixa é totalmente falso

- **Severidade:** Crítica
- **Módulo:** PDV Caixa
- **Tipo:** Frontend / Regra de negócio / Segurança
- **Evidência:** `src/pages/pdv/caixa/components/EstornoModal.tsx`
- Linha 6: `const SENHA_GERENTE MOCK = '1234';` — comparação `===` direta, sem validação de PIN/usuário real.
- `onConfirmar` (chamado de `PedidosRecentesPanel.tsx:1200,1663`) só executa `setEstornado(true)` local + `registrarEvento` (auditoria client-side).

- Tela de "sucesso" gera protocolo fake `EST${Date.now()}...` e textos como "Registrado no Log de Auditoria", "Valor estornado", "Autorizado por: Gerente".
- **Como reproduzir:** No PDV Caixa, abrir "Pedidos Recentes" → "Estornar" em qualquer pedido pago → digitar 1234 → ver tela de "Estorno Registrado".
- **Resultado esperado:** Reversão real do pagamento (`payments.is_refunded=true`), registro em `refunds`, ajuste em `cash_movements`/fluxo de caixa, e `audit_log`.
- **Resultado encontrado:** Nenhuma escrita no banco. A UI passa confiança total de que o dinheiro foi devolvido e o caixa ajustado.
- **Impacto:** Fraude interna trivial (qualquer um sabendo "1234" "estorna" e embolsa o valor em dinheiro, já que o sistema não registra a saída); divergência de caixa não detectável; "Estorno Registrado no Log de Auditoria" é mentira para o usuário.
- **Sugestão de correção:** Remover `EstornoModal.tsx` e a senha hardcoded; redirecionar a ação "Estornar" para o fluxo real (`fn_cancel_and_refund_order` via `src/components/feature/CancelamentoModal.tsx`), corrigindo antes o BUG-005 (restock).

BUG-002 — Quatro implementações distintas de cancelamento/reembolso de pedido, nenhuma completa

- **Severidade:** Crítica
- **Módulo:** PDV Caixa / Gestão de Pedidos
- **Tipo:** Backend / Banco / Regra de negócio
- **Evidência (confirmado via `pg_get_functiondef` no Supabase):** 1. `fn_cancel_order_bypass(order_id, user_id, reason)` — só seta `orders.status='cancelled'` e `order_items.status='cancelled'`. **Sem refund, sem restock, sem audit_log.** 2. `fn_cancel_and_refund_order(order_id, user_id, reason, restock=false)` — cancela pedido/itens, marca `payments.is_refunded=true`, insere em `refunds` (incluindo `restock_items=p_restock`), **mas NUNCA usa `p_restock` para de fato devolver estoque** — o campo é só metadado. 3. `order-write.cancel_order` (edge function) — refund/audit só ocorrem se o body trouxer `reason_type+refund_amount+refund_method`; restock só se além disso `restock_items=true`. `CancelOrderModal.tsx` (Gestor de Pedidos) **nunca envia esses campos**, então esse caminho na prática nunca faz refund/restock/audit pela Gestão de Pedidos. 4. `src/components/feature/CancelamentoModal.tsx` chama `fn_cancel_and_refund_order/fn_cancel_order_bypass/fn_restock_order` (RPC à parte, não inspecionada nesta rodada — nome sugere que faz o restock que falta nas outras três).
- **Como reproduzir:** Comparar o resultado de cancelar o mesmo tipo de pedido pago via (a) Gestor de Pedidos → "Cancelar este Pedido", e (b) PDV Caixa → "Pedidos Recentes" → cancelamento.
- **Resultado esperado:** Um único fluxo de cancelamento, com refund + restock (quando aplicável) + `audit_log` sempre.
- **Resultado encontrado:** Comportamento depende de QUAL TELA o operador usa — divergência de estado entre `payments`, `refunds`, `stock_movements` e `audit_log`.
- **Impacto:** Reconciliação de caixa e estoque fica dependente de qual botão o operador clicou; auditoria incompleta.
- **Sugestão de correção:** Unificar em uma única RPC (idealmente baseada em `fn_cancel_and_refund_order` + a lógica de `fn_restock_order`, se esta existir e estiver correta) chamada por todos os pontos de entrada (PDV, Gestor de Pedidos, Garçom).

BUG-005 — Cancelamento de pedido NUNCA devolve estoque, em nenhum caminho

- **Severidade:** Crítica
- **Módulo:** Estoque / Gestão de Pedidos / PDV
- **Tipo:** Backend / Banco / Regra de negócio
- **Evidência:** Nenhuma das funções `fn_cancel_order_bypass`, `fn_cancel_and_refund_order` (mesmo com `p_restock=true`), ou `order-write.cancel_order` (mesmo com `restock_items=true`, que só dispara se `reason_type+refund_amount+refund_method` também estiverem presentes) contém qualquer chamada a `fn_add_stock_movement/fn_update_ingredient_stock` do tipo `in` para reverter a baixa de `deductStockForOrderItem`.
- **Como reproduzir:** 1. Vender item com ficha técnica. 2. Avançar para "ready"/"delivered" (dispara baixa de estoque). 3. Cancelar o pedido por qualquer caminho, inclusive com "devolver ao estoque" marcado. 4. Conferir

`ingredients.current_stock` do insumo — não muda.

- **Resultado esperado:** Estoque do(s) insumo(s) consumido(s) por itens em status "ready"/"delivered" deve voltar ao `current_stock` quando o pedido é cancelado/estornado com restock.
- **Resultado encontrado:** Estoque permanece baixado permanentemente — toda venda cancelada após preparo gera divergência teórico×físico irreversível.
- **Impacto:** Acúmulo de divergência de estoque ao longo do tempo, distorce CMV, inventário, compras e relatórios de produção (módulo de semi-acabados é especialmente sensível, pois o custo de produtos derivados depende do estoque do insumo bruto).
- **Sugestão de correção:** Implementar restock real (provavelmente em `fn_restock_order`, se existir, ou criar `fn_restock_order_items`), inserindo `stock_movements` tipo `in` com `quantity = soma das deduções originais` para os `order_items` cancelados, e acioná-lo de forma consistente nos 4 caminhos do BUG-002.

BUG-010 — Status "Em Rota" (delivery) não existe no banco e nunca é persistido

- **Severidade:** Crítica
- **Módulo:** Gestão de Pedidos / KDS
- **Tipo:** Frontend / Banco / Regra de negócio
- **Evidência:**
- `src/pages/gestor-pedidos/page.tsx, handleEmRota` (~linha 344-353): apenas `setPedidos(prev => prev.map(...status:'em_rota'...))`.
- Confirmado no banco: `SELECT enum_range(NULL::order_status) → {draft, new, preparing, ready, delivered, cancelled}` — **em_rota não existe**.
- `GestorKanbanView.tsx` renderiza uma coluna "Em Rota" baseada nesse status puramente local.
- **Como reproduzir:** Marcar pedido de delivery como "Em Rota" → recarregar a página (F5) ou abrir em outro dispositivo.
- **Resultado esperado:** Pedido continua exibido como "Em Rota" após reload/em outro terminal.
- **Resultado encontrado:** Pedido volta para a coluna "Prontos" (DB só tem `ready`).
- **Impacto:** Operação de delivery não consegue rastrear quais pedidos já saíram para entrega entre dispositivos/turnos — risco de pedido "esquecido" ou entregador duplicando rota.
- **Sugestão de correção:** Adicionar valor `em_rota/out_for_delivery` ao enum `order_status` (`ALTER TYPE ... ADD VALUE`) e persistir via `fn_advance_order_status_safe` (ou variante), atualizando `handleEmRota` para chamar a RPC.

BUG-016 — Numeração de pedidos pode duplicar entre sessões simultâneas no mesmo dia

- **Severidade:** Crítica
- **Módulo:** Gestão de Pedidos / Numeração / Relatórios
- **Tipo:** Banco / Regra de negócio
- **Evidência (confirmado via `pg_get_functiondef`):** `fn_next_order_number(p_session_id, p_tenant_id): sql UPDATE sessions SET last_order_number = COALESCE(last_order_number,0)+1 WHERE id = p_session_id RETURNING last_order_number INTO v_seq; ... RETURN QUERY SELECT v_seq, ('P'||dd||mm||yy||LPAD(v_seq::text,4,'0'))`; O contador `last_order_number` é **por sessão** (`sessions.id`), começando em 1, e a data usada no número é `sessions.opened_at` (data de abertura da sessão). Se o sistema permite múltiplas `sessions` abertas simultaneamente para o mesmo tenant/loja no mesmo dia (ex.: caixa principal + caixa de delivery + autoatendimento, cada um abrindo sua própria sessão), **cada sessão gera, de forma independente, P<ddmmyy>0001, P<ddmmyy>0002, ...** — números de pedido **duplicados** entre sessões no mesmo dia.
- **Como reproduzir:** Abrir 2 sessões de caixa no mesmo dia (ex.: caixa 1 e autoatendimento), criar 1 pedido em cada — ambos recebem `P<ddmmyy>0001`.
- **Resultado esperado:** Numeração única por tenant/dia (ex.: contador centralizado por `tenant_id + data`, não por `session_id`).
- **Resultado encontrado:** Colisão garantida sempre que houver 2+ sessões abertas no mesmo dia — situação NORMAL neste sistema (caixa, autoatendimento, delivery podem operar em paralelo).
- **Impacto:** Impressão de comandas/comprovantes com número duplicado (confusão na cozinha/entrega/retirada por senha — já que `numero` também vira "senha" em alguns destinos), relatórios e buscas por número de pedido

ambíguos, possível colisão de chave caso `orders.number` tenha índice único por tenant+data.

- **Sugestão de correção:** Mudar a chave do contador de `session_id` para `(tenant_id, data_local)` — manter uma tabela/linha de sequência diária por tenant (ou usar uma `sequence/SELECT ... FOR UPDATE` numa tabela `daily_order_counters`). Esta é exatamente a categoria de bug que o usuário já havia sinalizado como risco histórico ("Numeração duplicada de pedidos entre sessões").

BUG-018 — Dois sistemas paralelos e incompatíveis de "pedido de mesa"

- **Severidade:** Crítica
- **Módulo:** Mesa QR / Mesa Cliente / Gestão de Pedidos
- **Tipo:** Backend / Integração / Banco
- **Evidência:**
 - `src/pages/mesa-qr/ (useMesaQRData.ts) → mesa-write.create_mesa_order` → grava orders com `origin_type='table', destination_type='password', destination_name = participant.access_token` (PIN sequencial), vinculado a `participant_id/table_session_id`.
 - `src/pages/mesa/page.tsx (MesaClientePage, linhas ~369-385) → order-write.create_order` **direto**, com `destination_type` mapeado por `DEST_MAP/ORIGIN_MAP`, `destination_name = clienteNome` (texto livre), **sem** `participant_id`, **sem validação contra table_session do mesa-write**.
- **Como reproduzir:** Comparar um pedido criado via `/mesa-qr/:qr_token` com um criado via `/mesa/:mesaId` para a mesma mesa — campos `destination_name/destination_type/vínculos` de sessão divergem completamente.
- **Resultado esperado:** Um único modelo de dados para "pedido de mesa", consumido de forma uniforme pelo Gestor de Pedidos, KDS, impressão (senha de retirada) e fechamento de conta.
- **Resultado encontrado:** Dois modelos coexistindo — qualquer lógica que assuma `destination_name` é um PIN (ex.: exibição de senha para chamada) quebra para pedidos vindos de `/mesa/:mesaId`, e vice-versa.
- **Impacto:** Inconsistência visível ao cliente final (senha de retirada não aparece/aparece errado dependendo de qual link ele usou), e nos relatórios de origem/mesa.
- **Sugestão de correção:** Decidir qual fluxo é o oficial (parece ser Mesa QR, conforme `project_plan.md` Fase 6) e migrar/depreciar `/mesa/:mesaId (MesaClientePage)` para usar o mesmo `mesa-write.create_mesa_order`, ou documentar e alinhar explicitamente os dois modelos.

ALTOS

BUG-006 — Estoque insuficiente não bloqueia venda e déficit real é mascarado

- **Severidade:** Alta
- **Módulo:** Estoque / PDV
- **Tipo:** Regra de negócio / Banco
- **Evidência:** `useEstoqueAlertaPDV.ts` (comentário explícito: "fail-safe, não bloqueia"); `fn_update_ingredient_stock/fn_add_stock_movement` usam `GREATEST(0, current_stock + delta)`.
- **Como reproduzir:** Insumo com `current_stock=1`, vender 2x um item que consome 1 unidade cada.
- **Resultado esperado:** Sistema impede a venda OU registra claramente "faltaram X unidades".
- **Resultado encontrado:** Ambas as vendas são aceitas; `current_stock` vai a 0 (nunca negativo) e `is_depleted=true`, mas não há registro de que faltou 1 unidade — a segunda baixa "desaparece" silenciosamente.
- **Impacto:** Gestor não sabe quanto realmente faltou; CMV/relatórios de consumo subestimam o uso real do insumo.
- **Sugestão de correção:** Decisão de produto: (a) bloquear/avisar fortemente no PDV antes de confirmar, e/ou (b) permitir `current_stock` negativo (remover o `GREATEST(0, ...)` ou criar um campo separado `shortfall_total`) para visibilidade real do déficit.

BUG-008 — Lotes (`ingredient_batches.quantity_remaining`) nunca são consumidos por vendas ou produção

- **Severidade:** Alta
- **Módulo:** Estoque

- **Tipo:** Backend / Banco
- **Evidência:** `fn_add_stock_movement` decrementa `ingredient_batches.quantity_remaining` **somente se** `p_batch_id` for informado e o tipo for `*_out`. Nem `deductStockForOrderItem` (order-write, vendas) nem `fn_register_production_and_stock_v2` (produção, que faz `UPDATE ingredients` direto sem passar por `fn_add_stock_movement`) informam `batch_id`.
- **Como reproduzir:** Cadastrar um lote com validade próxima e quantidade X; vender itens que consomem esse insumo; conferir `quantity_remaining` do lote — permanece igual a `quantity_received`.
- **Resultado esperado:** `quantity_remaining` reflete consumo real (idealmente FEFO — primeiro a vencer, primeiro a sair).
- **Resultado encontrado:** Tela de Validade/Lotes mostra quantidades desatualizadas, dando falsa sensação de estoque disponível por lote.
- **Impacto:** Risco sanitário/rastreabilidade (recall não consegue identificar lote consumido); planejamento de compras por validade fica incorreto.
- **Sugestão de correção:** Implementar seleção FEFO de lote em `deductStockForOrderItem` e passar `batch_id` para `fn_add_stock_movement`; idem em `fn_register_production_and_stock_v2`.

BUG-021 — `order-edit-lock.force_unlock` sem checagem de papel no servidor

- **Severidade:** Alta (Segurança)
- **Módulo:** Mesa / Edição de pedido
- **Tipo:** Segurança
- **Evidência:** `supabase/functions/order-edit-lock/index.ts:160-183` — comentário diz "Só admin/master pode usar — verificado pelo caller", mas a única checagem no servidor é pertencimento ao `tenant` via `user_tenants` (linhas 51-63), sem checagem de `perfil`.
- **Como reproduzir:** Usuário com perfil "garçom" (não admin/gerente) chama a action `force_unlock` para um pedido travado por outro usuário — servidor aceita.
- **Resultado esperado:** 403 se o usuário não for admin/gerente.
- **Resultado encontrado:** Qualquer usuário do `tenant` pode forçar destravamento.
- **Impacto:** Garçom pode interromper edição em andamento de outro usuário (gerente fazendo desconto, por exemplo), levando a perda de alterações ou condição de corrida na gravação do pedido.
- **Sugestão de correção:** Adicionar checagem de `perfil` `IN ('admin', 'gerente')` no handler `force_unlock` antes de liberar o lock.

BUG-022 — Dois algoritmos divergentes para "a mesa pode ser fechada?"

- **Severidade:** Alta
- **Módulo:** Mesas
- **Tipo:** Regra de negócio / Integração
- **Evidência:**
 - `MesasContext.fecharMesa` (`src/contexts/MesasContext.tsx:159-215`): verifica pendências via `payments.is_refunded=false`.
 - `table-write.close_table_by_customer` (linhas 107-166): verifica `totalPago >= totalConsumo * 0.99` (tolerância de 1%), independente de `is_refunded`.
- **Como reproduzir:** Criar cenário em que `totalPago` é 99,5% do `totalConsumo` com algum pagamento já estornado (`is_refunded=true`) — uma rota permite fechar, a outra não.
- **Resultado esperado:** Critério único de "conta fechada" (idealmente: soma de pagamentos não estornados $\geq$ total do pedido, sem tolerância arbitrária — ou com tolerância documentada e igual nos dois lados).
- **Resultado encontrado:** Funcionário (PDV/Garçom) e cliente (Mesa QR) podem ter permissões de fechamento diferentes para a mesma mesa no mesmo estado.
- **Impacto:** Mesa pode ficar travada para um lado e liberada para o outro — confusão operacional, possível fechamento com saldo devedor de até 1% sem aviso.
- **Sugestão de correção:** Extrair a checagem para uma função única (RPC) usada por ambos os caminhos.

BUG-025 — Pedido do Autoatendimento sem sessão de caixa não persiste no banco

- **Severidade:** Alta
- **Módulo:** Autoatendimento
- **Tipo:** Frontend / Integração
- **Evidência:** `src/pages/autoatendimento/page.tsx:505-541`, branch `!sessionId`: chama apenas `addPedido(buildKDSPedido(...))` (estado local/KDS local) — nenhuma chamada a `order-write`.
- **Como reproduzir:** Iniciar pedido no totem sem caixa aberto (ou com `sessionId` indisponível) e finalizar.
- **Resultado esperado:** Pedido é criado no banco (eventualmente vinculado a uma sessão quando uma for aberta) ou o totem bloqueia o uso até haver sessão ativa.
- **Resultado encontrado:** Pedido "existe" apenas na tela local, **desaparece em F5**, e `pedidoSeq` (contador local, módulo-level) reseta para 1000 ao recarregar — risco de números de pedido locais colidirem.
- **Impacto:** Pedido pago/confirmado pelo cliente pode simplesmente sumir; cozinha nunca recebe.
- **Sugestão de correção:** Bloquear o totem (tela de "fora de serviço") quando não houver sessão de caixa ativa, em vez de permitir pedido "fantasma" local.

BUG-026 — Fallback de formas de pagamento do Kiosk usa IDs não-UUID

- **Severidade:** Alta
- **Módulo:** Autoatendimento
- **Tipo:** Frontend / Integração
- **Evidência:** `FormaPagamentoKiosk.tsx` — lista de fallback usa IDs como `'pix'`, `'credito'`, `'debito'`, `'dinheiro'`. `payment_method_id` é FK para `payment_methods.id` (uuid) em `fn_record_payment_bypass`.
- **Como reproduzir:** Forçar falha no fetch de `payment_methods` (offline/erro) e finalizar pedido pagando na hora.
- **Resultado esperado:** Pagamento gravado normalmente ou pedido cai para "pagar na entrega".
- **Resultado encontrado:** `record_payment` provavelmente falha (FK inválida) — erro pouco claro para o cliente no totem.
- **Sugestão de correção:** Fallback deve usar UUIDs reais de formas de pagamento padrão (cacheadas/seedadas), nunca strings literais.

BUG-037 — CMV calculado de duas formas diferentes (Relatórios vs Estoque)

- **Severidade:** Alta
- **Módulo:** Relatórios / Estoque
- **Tipo:** Regra de negócio / Integração
- **Evidência:** `useCmvRelatorio.ts` (RPC `fn_get_cmv_report`, base = `orders.total_amount`) vs `useCmvReport.ts` (estoque, base = `SUM(order_items.item_price * quantity)`), com filtro de status potencialmente diferente).
- **Como reproduzir:** Abrir Relatórios → CMV e Estoque → CMV "Realizado" para o mesmo período e comparar % de CMV.
- **Resultado esperado:** Mesmo número.
- **Resultado encontrado:** Provavelmente diferente (bases de receita diferentes: total do pedido vs soma de itens).
- **Impacto:** Credibilidade do sistema de relatórios — gestor vê dois CMVs diferentes na mesma ferramenta.
- **Sugestão de correção:** Unificar em `fn_get_cmv_report` como fonte única; remover/realinhar a agregação client-side de `useCmvReport.ts`.

MÉDIOS

BUG-007 — Possível inconsistência em `confirm_inventory` (estoque)

- **Severidade:** Média | **Módulo:** Estoque | **Tipo:** Backend
- `stock-write.confirm_inventory` chama `fn_add_stock_movement` por item com diferença (que já aplica `current_stock = GREATEST(0, current_stock+delta)`) e, em seguida, sobrescreve `current_stock = qtdContada` diretamente para todos os itens contados. Resultado final correto em condições normais, mas se

`fn_add_stock_movement` falhar no meio do loop (linha apenas faz `console.error` e continua), o `stock_movement` registrado fica dessincronizado do `current_stock` final.

- **Sugestão:** Tratar falha de `fn_add_stock_movement` como erro bloqueante (abortar/realimentar o usuário), não apenas logar.

BUG-009 — Custo/rendimento de produção calculado no frontend sem validação server-side

- **Severidade:** Média | **Módulo:** Estoque/Produção | **Tipo:** Backend/Segurança
- `fn_register_production_and_stock_v2` recebe `p_total_cost/p_unit_cost` prontos do frontend (RegistroProducaoModal.tsx), sem recálculo/validação contra `ingredients.unit_price` no momento da chamada.
- **Sugestão:** Validar/recalcular no banco; rejeitar se divergência > tolerância.

BUG-011 — `fn_advance_order_status_safe` não diferencia status por item nem atualiza timestamps de SLA

- **Severidade:** Média/Alta (depende de uso real de multi-estação) | **Módulo:** KDS | **Tipo:** Backend
- A função (confirmada via `pg_get_functiondef`) faz `UPDATE order_items SET status=p_status WHERE order_id=... AND status!='cancelled'` — ou seja, **todos os itens não cancelados mudam juntos**, e a função **não grava** `started_preparing_at/ready_at/delivered_at`. Se o frontend depende desses timestamps para SLA (`SlaCell`, `pedido_item_partes`), e nenhuma outra rotina os preenche, os cálculos de SLA por item ficam sem dados reais.
- **Sugestão:** Confirmar se há outra função/trigger preenchendo esses timestamps; se não, adicionar à RPC. Avaliar suporte a status por item para multi-estação.

BUG-012 — Cancelamento simples não gera entrada de auditoria

- **Severidade:** Média/Alta | **Módulo:** Gestão de Pedidos | **Tipo:** Regra de negócio
- `PedidoEditHistory.tsx` filtra `action_type` IN ('order_edit_started', 'order_edit_finished', 'order_edit_force_unlocked', 'discount_applied', 'order_refunded'). Como `fn_cancel_order_bypass` não grava `audit_log`, e `order-write.cancel_order` só grava `order_refunded` no bloco condicional (raramente atingido pela UI — ver BUG-002), **cancelamentos via Gestor de Pedidos ficam invisíveis no histórico de auditoria do pedido**.
- **Sugestão:** Gravar `audit_log` (`action_type='order_cancelled'`) sempre que um pedido é cancelado, independente de refund.

BUG-013 — `kdsParaRecente()` zera valores financeiros dos itens

- **Severidade:** Média | **Módulo:** Pedidos | **Tipo:** Frontend
- `src/pages/pedidos/components/utils.ts:104,121` — `total: 0, preco: 0` hardcoded ao converter pedidos vindos do KDS para o formato "PedidoRecente". Se essa função alimenta `PedidosMetricas/SlaCell` para pedidos provenientes do KDS local, faturamento/ticket médio desses pedidos aparece zerado.
- **Sugestão:** Confirmar pontos de uso de `kdsParaRecente` em `src/pages/pedidos/page.tsx` e preencher `preco/total` reais a partir de `order_items`.

BUG-014 — Três definições diferentes de threshold de SLA

- **Severidade:** Média | **Módulo:** Pedidos/KDS | **Tipo:** Frontend
- `utils.ts` (`slaAlvo:15, atrasado: slaCozinha>15`) vs `SlaCell.CozinhaTag` (`alerta ≥10, crítico ≥15`) vs `SlaCell.EsperaTag` (`alerta ≥7, crítico ≥12`) — sem equivalente em `utils.ts`.
- **Sugestão:** Centralizar thresholds de SLA em uma única constante/config (idealmente vinda de `estacoes_cozinha.sla_minutos` por estação, conforme `project_plan.md`).

BUG-017 — Cancelamento de pedido pago não verifica `isPaid` na Gestão de Pedidos

- **Severidade:** Média | **Módulo:** Gestão de Pedidos | **Tipo:** Frontend/Regra de negócio
- `PedidoDetailModal.tsx:404` — botão "Cancelar este Pedido" aparece para qualquer pedido não cancelado/não entregue, **sem verificar `pedido.isPaid`**. Combinado ao BUG-002/005, cancelar um pedido pago e não entregue gera pagamento órfão sem refund automático.
- **Sugestão:** Se `isPaid=true`, exigir fluxo de cancelamento+reembolso (não o cancelamento simples).

BUG-019 — Pelo menos 3 vocabulários de status de mesa coexistindo

- **Severidade:** Média | **Módulo:** Mesas | **Tipo:** Banco/Frontend
- `mesa-write` usa `available/occupied`; `table-write.close_table_by_customer` seta `'livre'`; `MesasContext` tipa `Mesa.status` como `'livre'` | `'ocupada'` | `'reservada'` | `'bloqueada'` e também checa `'blocked'`.
- **Sugestão:** Padronizar enum único de status de mesa no banco e usar consistentemente.

BUG-039/040 — Fallback de `useCaixaReport` zera dados financeiros sem aviso

- **Severidade:** Média (a RPC principal está correta; risco só no fallback) | **Módulo:** Relatórios | **Tipo:** Frontend
- Quando `fn_get_cash_sessions_v2` falha/retorna vazio, `useCaixaReport.ts` monta um objeto com `total_descontos:0`, `total_troco:0`, `cash_transactions:[]`, `movimentos.lista:[]` — sem flag `is_fallback` para a UI avisar "dados parciais".
- **Sugestão:** Adicionar flag de modo degradado e popular esses campos a partir de `orders/payments/cash_movements` mesmo no fallback.

BUG-041 — `porOrigem` (RPC) vs `porHora` (query direta) podem divergir

- **Severidade:** Média | **Módulo:** Relatórios | **Tipo:** Integração
- `useOrigemReport.ts` usa `fn_get_sales_report.by_destination` para totais por origem, mas `porHora` vem de query direta com filtro próprio (`status not in (cancelled,draft)`, `is_training=false`, `is_draft=false`) — se a RPC usa critério diferente (ex.: também exclui cortesias), os totais não batem.
- **Sugestão:** Unificar fonte ou alinhar filtros explicitamente.

BAIXOS / MELHORIAS

ID	Descrição	Módulo
BUG-020	<code>participant_name</code> enviado em <code>create_mesa_order</code> mas nunca usado pelo <code>mesa-write</code> (campo morto)	Mesa QR
BUG-023	Encerramento de mesa via <code>/mesa/:mesaId</code> não limpa <code>localStorage</code> keys do fluxo Mesa QR (<code>mesa_participant_*</code>)	Mesa
BUG-024	<code>MesaClientePage</code> "fail-open" em erro de fetch de status da sessão (assume <code>'open'</code>)	Mesa
BUG-027	Sem tratamento amigável para erro 403 <code>session_closed</code> no Kiosk	Autoatendimento
BUG-028	Comanda principal de cozinha (<code>printKitchenTicket</code>) usa caminho direto sem fila/retry — perda silenciosa se impressora+agente offline	Impressão
BUG-029	<code>print-queue-write/index.ts</code> é edge function órfã (schema próprio, nunca chamada pelo frontend, que usa RPC <code>enqueue_print_ticket</code>)	Impressão
BUG-030	Tickets podem ficar presos em <code>status='printing'</code> para sempre se o agente cair entre poll e confirm (sem timeout de recuperação)	Impressão
BUG-031	Risco de impressão duplicada se <code>queueOrderForPrint</code> e <code>printKitchenTicket</code> forem ambos disparados para o mesmo pedido (não confirmado se ocorre)	Impressão

BUG-032	Backup automático de impressora pode imprimir 2 vias em caso de falso-timeout	Impressão
BUG-033	Resolução "primeira impressora de rede" como fallback silencioso (<code>console.warn</code> apenas)	Impressão
BUG-034	Tabela de conversão CP860 duplicada em 3-4 arquivos	Impressão
BUG-035	<code>printSimpleReceipt</code> e <code>ComprovantePrint</code> geram layouts ligeiramente divergentes para o "mesmo" comprovante	Impressão
BUG-038/042	<code>useDeliveryReport</code> reimplementa cálculo de datas Brasília em vez de usar <code>dateUtils.getPeriodDates</code> ; timezone fixo -03:00 em todo o sistema	Relatórios

5. Inconsistências Frontend/Backend

- Cancelamento:** 4 implementações (BUG-002) com nomes de parâmetros e efeitos colaterais diferentes (`restock_items` vs `p_restock` — ambos ignorados na prática).
- CMV:** `useCmvRelatorio.CmvItem` (`total_qty`, `tem_ficha_tecnica`) vs `useCmvReport.CmvItemVendido` (`qtd_vendida`, `tem_ficha`) — mesma informação, nomes diferentes (BUG-037).
- Origem do pedido:** coluna `orders.origin_type` vs campo `destination` retornado por `fn_get_sales_report.by_destination` — nomenclatura trocada entre banco e RPC.
- Status de mesa:** `available/occupied` (mesa-write) vs `livre/ocupada/reservada/bloqueada` (frontend) vs `blocked/livre` (table-write) — BUG-019.
- Status de pedido "Em Rota":** existe no frontend (Kanban, botão) mas não no enum do banco — BUG-010.
- Movimentação de estoque:** `MOVEMENT_TYPE_MAP` (stock-write, PT→EN) vs `order-write` que grava tipos em inglês diretamente — duas portas de entrada para o mesmo enum `stock_movement_type`, frágil a mudanças.
- `print-queue-write`** define schema próprio para `print_queue` divergente do que a RPC `enqueue_print_ticket` realmente usa (função órfã) — BUG-029.
- Numeração de pedido:** formato `Pddmmaa0001` é gerado corretamente pela RPC, mas o fallback client-side `P${Date.now()}` (se a RPC retornar vazio) quebra esse formato — usado por gerarNumeroStr/formatOrderNumber em outros lugares.`

6. Riscos Financeiros

Risco	Causa raiz	Bugs relacionados
Estorno fraudulento sem rastro	<code>EstornoModal</code> fake, senha hardcoded	BUG-001
Pagamento de pedido cancelado fica "perdido" sem refund	Cancelamento via Gestor de Pedidos não envia campos de refund	BUG-002, BUG-012
Estoque não devolvido em cancelamentos	Nenhuma RPC implementa restock de fato	BUG-005, BUG-008
Numeração de pedido duplicada → comprovantes/comandas/senhas conflitantes, possível erro de entrega errada (mesmo número, dois pedidos diferentes)	Contador por sessão, não por tenant/dia	BUG-016
Dois números de CMV diferentes → decisões de precificação baseadas em dado errado	Bases de cálculo diferentes	BUG-037

Fechamento de caixa "zerando" desconto/troco no fallback → diferença de caixa não explicada	Fallback de <code>useCaixaReport</code>	BUG-039/040
Custo de produção (semi-acabados) manipulável/sem validação → CMV em cascata errado	<code>fn_register_production_and_stock</code> confia no frontend	BUG-009
Pedido do totem "some" sem sessão de caixa → venda perdida ou cliente pagou e pedido não chega à cozinha	Modo offline do Autoatendimento	BUG-025

7. Riscos para Produção (bloqueadores obrigatórios)

Antes de operar com cliente real, classificar como **obrigatórios**:

1. **BUG-001** — Remover/corrigir `EstornoModal` (estorno fake com senha fixa). Risco de fraude/perda financeira direta.
2. **BUG-016** — Corrigir numeração de pedidos para ser única por tenant/dia (não por sessão). Sem isso, qualquer operação com 2+ terminais simultâneos (caixa + autoatendimento, por exemplo — cenário padrão deste sistema) já gera números duplicados no dia 1 de uso real.
3. **BUG-002 / BUG-005** — Unificar e completar o fluxo de cancelamento/reembolso/restock. Sem isso, todo cancelamento pós-pagamento ou pós-preparo gera divergência financeira e de estoque.
4. **BUG-018** — Decidir e unificar o fluxo de "pedido de mesa" (Mesa QR vs Mesa Cliente) antes de divulgar QR Codes para clientes reais — caso contrário, comportamento de senha/conta dependerá de qual link foi usado.
5. **BUG-025 / BUG-026** — Garantir que o Autoatendimento nunca aceite pedido sem persistência real (bloquear uso sem sessão de caixa válida e formas de pagamento válidas).
6. **BUG-028/030/031** — Cadeia de impressão da cozinha precisa de fila com retry confiável (hoje a comanda principal não tem) — pedido pago sem comanda impressa = pedido não preparado.

Itens **importantes mas não bloqueadores absolutos** (podem ir em paralelo/curto prazo pós-lançamento piloto):
BUG-006 a BUG-014, BUG-017, BUG-019, BUG-021, BUG-022, BUG-037 a BUG-042.

8. Avaliação da Cobertura de Testes/Simulações Existentes

- **qa-full-simulation**: roda em tenant QA isolado; cobre criação multi-cenário, avanço de status (preparing/ready/delivered via `fn_advance_order_status_safe`) e 2 cancelamentos — mas **exclui pedidos cancelados da validação cruzada**, mascarando exatamente o BUG-005 (estoque não devolvido) e BUG-002/012 (sem refund/audit).
- **simulate-orders**: insere `orders/order_items` **diretamente via `.insert()`**, ignorando `order-write`/RPCs reais — não detecta regressões nas funções auditadas aqui (incluindo BUG-016, já que não chama `fn_next_order_number`).
- **simulate-pdv-orders**: 12 cenários reais via RPC, mas **roda no tenant de PRODUÇÃO real** (9063797b-a50b-4d9a-ac09-d232ddcd48d1) e nunca avança status além de `new` — não cobre KDS/cancelamento. Risco operacional: pedidos `SIM-*` poluindo dados reais (QA-10).
- **ChecklistTeste.tsx**: 100% manual, sem persistência de evidência.
- **Testes unitários (`src/test/business`, `src/test/integration`)**: cobrem `orderLogic/statusMappers/fluxo` de pedido em memória/mock — não exercitam as RPCs reais, portanto não pegam nenhum dos bugs de banco confirmados aqui (BUG-002, BUG-005, BUG-008, BUG-016).

Conclusão: as simulações existentes validam bem o "caminho feliz" de criação/pagamento, mas têm **zero cobertura** para cancelamento completo (refund+restock+audit), status "Em Rota", numeração entre sessões concorrentes, e os dois fluxos de mesa. O QADashboard reporta "bugs corrigidos" históricos com testes hardcoded — não generaliza para os bugs novos encontrados nesta auditoria, podendo dar falsa sensação de saúde do sistema.

9. Plano de Teste Final Sugerido (para quando testes automatizados forem permitidos)

Bateria 1 — Cancelamento/Reembolso/Restock (prioridade máxima)

1. Criar pedido, pagar, avançar item até "ready" (gera baixa de estoque), cancelar com `refund+restock=true` → verificar `payments.is_refunded`, `refunds`, `ingredients.current_stock` (deve voltar), `audit_log`.
2. Repetir via os 4 caminhos (PDV/EstornoModal removido, CancelamentoModal, Gestor de Pedidos, fluxo de garçom) e comparar resultado — devem ser idênticos.
3. Cancelar pedido **antes** do pagamento — não deve gerar `refunds`.

Bateria 2 — Numeração concorrente

4. Abrir 2 sessões de caixa simultâneas (tenant teste) no mesmo dia, criar 1 pedido em cada, verificar `orders.number` — não pode haver duplicata.
5. Criar 5 pedidos intercalados entre as 2 sessões, verificar sequência e formato `Pddmmaa00xx`.

Bateria 3 — Mesa QR vs Mesa Cliente

6. Criar pedido via `/mesa-qr/:qr_token` e via `/mesa/:mesaId` para a mesma mesa física — comparar `destination_name/destination_type`/exibição de senha no Gestor de Pedidos e KDS.
7. Testar fechamento de mesa pelos dois critérios (BUG-022) com pagamento parcial (99%-100%) e com pagamento estornado.

Bateria 4 — Status "Em Rota" / KDS

8. Marcar pedido delivery como "Em Rota", recarregar página/abrir em outro terminal — deve persistir (após correção do BUG-010).
9. Pedido multi-estação: avançar apenas uma estação para "ready" — verificar se `order_items` de outras estações permanecem com status próprio (após correção do BUG-011).

Bateria 5 — Estoque / Produção

10. Vender item até `current_stock=0`, vender novamente — verificar comportamento (bloqueio ou registro de déficit, conforme decisão do BUG-006).
11. Criar lote, vender itens que o consomem, verificar `ingredient_batches.quantity_remaining` (após correção do BUG-008).
12. Registrar produção de semi-acabado com custo/rendimento, alterar payload manualmente (DevTools) e verificar se o backend rejeita/recalcula (BUG-009).

Bateria 6 — Impressão

13. Desligar agente local + impressora, finalizar pedido com itens de cozinha — verificar se a comanda entra em fila com `retry` (após correção do BUG-028).
14. Forçar timeout no envio TCP (simular rede lenta) — verificar se não há impressão duplicada (BUG-032).

Bateria 7 — Relatórios

15. Comparar CMV de Relatórios vs Estoque para o mesmo período (após unificação do BUG-037).
16. Forçar falha da RPC `fn_get_cash_sessions_v2` (ex.: parâmetro inválido) e verificar se o fallback do `useCaixaReport` ainda mostra desconto/troco/transações reais (após correção do BUG-039/040).

Bateria 8 — Autoatendimento

17. Simular totem sem sessão de caixa ativa — deve bloquear pedido, não criar pedido "fantasma" local (BUG-025).
18. Simular falha no fetch de `payment_methods` — verificar que nenhum `payment_method_id` inválido é enviado (BUG-026).

Observação metodológica

As funções SQL/RPC foram verificadas diretamente no banco de produção via `pg_get_functiondef` (projeto Supabase `mdghhjzmdmeuqzuyzx`, "ERP OS") em modo somente leitura (`SELECT`), já que o repositório não contém arquivos de migration (`supabase/migrations` não existe). Itens marcados "■■■ Precisa validar manualmente" referem-se a RPCs não verificadas nesta rodada (`fn_get_sales_report`, `fn_get_cmv_report`, `fn_get_cancelamentos_report`, `fn_restock_order`, `fn_record_payment_bypass`, `fn_check_stock_alert_for_items`) — recomenda-se repetir esta consulta para essas funções na próxima rodada.

Nenhum arquivo de código ou dado em produção foi alterado durante esta auditoria.